Just Ask: An Interactive Learning Framework for Vision and Language Navigation

Ta-Chung Chi*
Carnegie Mellon University
tachungc@andrew.cmu.edu

Mihail Eric
Alexa AI
mihaeric@amazon.com

Seokhwan Kim
Alexa AI
seokhwk@amazon.com

Minmin Shen
Alexa AI
shenm@amazon.com

Dilek Hakkani-tur
Alexa AI
hakkanit@amazon.com

Abstract

In the vision and language navigation task Anderson et al. [2018], the agent may encounter ambiguous situations that are hard to interpret by just relying on visual information and natural language instructions. We propose an interactive learning framework to endow the agent with the ability to ask for users' help in such situations. As part of this framework, we investigate multiple learning approaches for the agent with different levels of complexity. The simplest model-confusion-based method lets the agent ask questions based on its confusion, relying on the predefined confidence threshold of a next action prediction model. To build on this confusion-based method, the agent is expected to demonstrate more sophisticated reasoning such that it discovers the timing and locations to interact with a human. We achieve this goal using reinforcement learning (RL) with a proposed reward shaping term, which enables the agent to ask questions only when necessary. The success rate can be boosted by at least 15% with only one question asked on average during the navigation. Furthermore, we show that the RL agent is capable of adjusting dynamically to noisy human responses. Finally, we design a continual learning strategy, which can be viewed as a data augmentation method, for the agent to improve further utilizing its interaction history with a human. We demonstrate the proposed strategy is substantially more realistic and data-efficient compared to previously proposed pre-exploration techniques.

1 Introduction

Consider the situation in which you want a robot assistant to get your wallet on the bed as in Figure 1 with two doors in the scene and an instruction that only tells it to walk through the doorway. In this situation, it is clearly difficult for the robot to know exactly through which door to enter. If, however, the robot is able to discuss the situation with the user, the situational ambiguity can be resolved. For example, the agent can ask the user "I am confused, please tell me which door to take?" and displays a snapshot on the user's smartphone of what it sees through its camera. The agent can then decide its next action by also considering the user's response.

This scenario suggests that interactive robots can get simple advice from their users to improve completion of tasks, in contrast to their passive counterparts that have no way of getting feedback when problems occur. Indeed, we note that the recent success of virtual assistants can be attributed to

*Work done while the first author was an intern at Alexa AI.

33rd Conference on Neural Information Processing Systems (NeurIPS 2019), Vancouver, Canada.



Figure 1: This is a real example from our trained agent. The instruction is *"...Walk straight, right before you reach the bed."*. The navigable locations are visualized by blue cylinders. It is impossible to determine which bedroom to enter, and our proposed agent asks for help in this situation.

their interactive ability with users, demonstrating several human-alike behaviors, such as asking for more information, clarification, and confirmation, which is useful for resolving ambiguities arising naturally in real-world tasks. Unfortunately, we do not notice such interactive behavior in physical robots. To the best of our knowledge, existing works MacMahon et al. [2006], Chen and Mooney [2011], Blukis et al. [2018], Anderson et al. [2018], Chen et al. [2019] required the robot to complete tasks by itself after the input of preliminary goals and instructions. It has no way to resolve confusions or ambiguities while executing its task, motivating this work's proposed interactive framework. We use the term *robot* and *agent* interchangeably hereafter since our robot lives in a simulator, which can be viewed as a virtual agent.

We propose to extend the Vision and Language Navigation task (VLN) Anderson et al. [2018] that evaluates how well an agent can learn to navigate in an indoor environment and reach a target location by following natural language instructions provided by a human user. To achieve this goal, the agent has to simultaneously understand visual surroundings and natural language. The inherently ambiguous natural language instructions and the complexity of the environment may jointly cause confusion and impede the robot's progress. In addition to the previous two doors example (shown as Figure 1), we observe many vague instructions in the VLN dataset, such as "walk a bit", where the distance to walk is not clear. In both cases, an agent cannot determine the correct action to execute by only relying on the visual cues and natural language instructions.

Recent approaches to address these difficulties can be mainly characterized into two directions: the first is to explore a better learning framework such that the agent rolls back to previous states when it is confused Wang et al. [2019], Ma et al. [2019a], Ke et al. [2019], and the second is to rely on semi-supervised data augmentation Fried et al. [2018], Tan et al. [2019]. In the first line of research, the length of traversed trajectories tend to be much longer than necessary since the agent has to explore the unknown by itself. On the other hand, the data augmentation approach suffers from several problems including: 1) Previous methods sample a lot of trajectories in the test environment, which would be time and resource inefficient if it were pursued by a robot in real life and 2) these approaches generate augmented data in a user-agnostic way, such that the agent cannot bootstrap off of user-specific patterns. With these drawbacks in mind, we ask what a real human would do in such scenarios with insufficient information. The answer is simple – *just ask for help*.

To investigate the interactive behavior in a principled way, we propose three critical aspects for a learning framework:

- *Temporal Resolution* - What is the ideal timing of interactions in the agent action sequence?
- *Question Category* - What type of question should be asked? (i.e. request, disambiguation, confirmation, etc.)
- *Interaction Form* - How to properly formulate agent questions and human responses for the communication to be efficient?

In this work, for *Temporal Resolution*, the timing is learned either naturally during the navigation or by leveraging human expert knowledge. For *Question Category*, we focus on the request question, namely "Which action should the robot take next, amongst the possible next actions?". For *Interaction Form*, we always use the previous request question, and the human response is the correct next action in the agent's action space. Other types of questions, such as "Shall I turn left?" (confirmation), "Shall I turn left or right?" (disambiguation), and how to effectively generate responses in natural language are left as future work.

Two agent models are proposed in this work. The simpler model, called *Model Confusion* or *MC*, mimics human user behavior under confusion. The more complicated model, called *Action Space Augmentation* or *ASA*, is an RL agent with the action space designed specifically to include questions.

It automatically learns to ask only necessary questions at the right timing during the navigation thanks to a proposed reward shaping mechanism. To better simulate real-world noise, we design a realistic way to distort answers given by users, while only the high-level RL agent adapts dynamically to different levels of noise.

We find that while the second agent achieves a high success rate, it still struggles with the problem of asking questions in similar situations. To address this concern, we gather the human-agent interaction data, which is used to fine-tune the agent further such that it gets familiar with the environment.

Overall, the main contributions of our work are four-fold:

- We are among the first to introduce human-agent interaction in the instruction-based navigation task, focusing on successful task completion with minimum questions to users.
- We propose two interaction methodologies, *MC* and *ASA*, that allow the agent to benefit from human-in-the-loop learning.
- We design a simulated user for responding to agent questions and propose alternative ways of creating realistic response data.
- We use the proposed approach for data augmentation, which is useful in a continual learning scenario, such that the agent can improve its performance continually in customers' home.

2 Related Work

Instruction-Based Navigation The instruction-based navigation tasks that use natural language and vision to perform robot navigation have been investigated extensively, including works done in synthetic environments MacMahon et al. [2006], Chen and Mooney [2011], Blukis et al. [2018], or agents trained in photo-realistic environments Anderson et al. [2018], Savva et al. [2019], Chen et al. [2019]. The VLN task Anderson et al. [2018] has received significant attention recently. Several works in this line designed more powerful agents by using panoramic views Fried et al. [2018], a better exploration strategy Ma et al. [2019a], Ke et al. [2019], Ma et al. [2019b], or generating more diverse environments as training data Tan et al. [2019]. Wang et al. [2019] proposed the cross-modal intrinsic reward for better training. However, due to the lack of interaction ability, the best strategy used by these works for the situation in Figure 1 would only be a random guess.

Human Robot Interaction Human robot interaction has long been investigated in the artificial intelligence field Goodrich et al. [2008], specifically using dialogue as the interaction format to complete physical tasks Lopes and Teixeira, Spiliotopoulos et al., Fong et al. [2003]. Recently, an end-to-end pipeline was presented Thomason et al. [2019a] for translating natural language commands to discrete robot actions. Clarification dialogues are used to improve language parsing and concept grounding. However, the dialogues only take place before the navigation process, ignoring the possibility that confusions may arise throughout the navigation. Another work Thomason et al. [2019b] proposed to integrate human-agent interaction by introducing dialogue behavior into the VLN task. The main contribution of their work is a human-to-human dialogue dataset for the navigation task, where two crowd workers are asked to complete the navigation task by interacting with each other. We foresee that the interaction dynamic between two human might be different from that between a human and an agent, that is, agent questions do not necessarily have to be based on human-human confusions, and instead should be based on the modeling approach used and the confusions of the models. Furthermore, the ultimate task in our opinion is successful task completion during navigation. Hence our proposed framework mainly focuses on when and what to ask for optimizing task success, with a minimum additional load on the user (i.e. the agent is expected to learn to ask the minimum or strictly necessary number of questions).

Data Augmentation Data augmentation has been shown to be an effective way to further boost the performance of the agent as pointed out in Fried et al. [2018], Ma et al. [2019b], Tan et al. [2019]. The common approach, borrowing inspiration from unsupervised machine translation Sennrich et al. [2015], Lample et al. [2017], is to sample some trajectories in the environment and use the speaker model Fried et al. [2018] to generate fake natural language instructions. They benefit from their unsupervised nature since there is no need for human effort. However, the amount of the augmented data is very large, which is only feasible during the training phase assuming that we have a simulator. Others Wang et al. [2019], Tan et al. [2019] extended this strategy to the unseen/test environment, but the agent still has to explore the environment for a large number of turns, which is too energy-inefficient and slow for the real world. Moreover, the agent cannot learn user-specific characteristics using the generated and fake instructions. In contrast, the human-agent interaction we propose can be used to generate augmented data naturally and efficiently, which only needs human to answer a few

questions. Since the instructions are generated by human, they are user-specific by nature. This is particularly useful when the goal is to let a human teach the agent with minimum effort.

3 Problem Formulation

Our task is the room-to-room navigation problem Anderson et al. [2018]. The agent is given a natural language instruction with l words, $I = \{w_1, w_2 \cdots w_i\}_{i=1}^{l}$, where each w_i is a word token. This instruction describes the navigation route R, which is represented by a sequence of n viewpoints, $R = \{v_1, v_2 \cdots v_j\}_{j=1}^{n}$. We use the terms *location* and *viewpoints* interchangeably hereafter, because the simulator used in this work does not support continuous movement between different viewpoints, so the location of the agent must be on a viewpoint. The agent starts from the start location v_1, going to the target location v_n. Since determining when to *stop* is critical, we denote the final location that the agent decides to stop at as v_f.

We formulate this navigation problem as a Markov Decision Process (MDP). At time step t, the state of the agent is s_t and the possible action space is $\{a_k^t\}_{k=1}^{m}$. Formally, the state s_t can be represented by the agent's 36-discretized panoramic view p_i^t, and its corresponding horizontal heading θ_i^t and vertical elevation ϕ_i^t:

$$f_i^t = \{Res(p_i^t), sin(\theta_i^t), cos(\theta_i^t), sin(\phi_i^t), cos(\phi_i^t)\}, \tag{1}$$

where i is between 1 to 36 and $Res(p_i^t)$ is a feature vector derived from a ResNet He et al. [2016] pretrained on imagenet Deng et al. [2009]. For each of the 36 viewing angles, the environment provides the corresponding next navigable locations. The collection of these locations and a *stop* action to end the navigation constitute the action space $\{a_k^t\}_{k=1}^{m}$, where each location is also represented by Eq. (1). Note that m may not be the same for different time step t. This number is determined by the simulator, since the agent may be blocked by obstacles at some of the 36 viewing angles. Once the agent picks an action $a^t \in \{a_k^t\}_{k=1}^{m}$, the agent moves to a new location with state s_{t+1}. To solve the MDP, our baseline model follows the previous idea Tan et al. [2019], which leverages imitation learning and RL techniques.

3.1 Interaction Ability

The agent may encounter ambiguities or get lost during the navigation. Therefore, it is desirable to endow the agent with the ability to interact with a human. Whenever the agent is confused, it sends out a signal "*I am lost, please help me!*" to the simulated user and asks for help. We assume the simulated user is an oracle O, which means it knows where the agent is and returns the next shortest path action to v_n. However, this is only feasible in the simulated environment because real users make mistakes when giving step-wise instructions due to various reasons, including the complex 3D environment. To simulate this test time error, we assume that users make mistakes with probability C. In this case, we calculate the angular differences between the shortest path action θ_{sh} and the remaining $m-1$ ones, which are then normalized by a softmax function. We do not sample randomly because even when users make mistakes, we assume that they are more likely to provide actions close to the shortest path action. Formally, $softmax_{i \neq sh}(-|\theta_{sh}^t - \theta_i^t|)$. The distorted answer is sampled from this distribution.

4 Model

4.1 Architecture

Our base model architecture is inspired by previous work in VLN Fried et al. [2018], Tan et al. [2019]. Each word of the instruction is represented by its word vector, $\tilde{w}_i = Embedding(w_i)$. A bidirectional LSTM model is used to encode the instruction, $\{u_i\}_{i=1}^{l} = LSTM(\{\tilde{w}_i\}_{i=1}^{l})$, where u_i is the concatenation of forward and backward output. The attentive panoramic view serves as the visual input, $\tilde{f}_i^t = \sum_{i=1}^{36} \alpha_i^t f_i^t, \alpha_i^t = softmax_i(f_i^t W_f \tilde{h}^{t-1})$, where $\tilde{h}^{t-1}$ is the previous instruction-aware hidden state, and W_f is a learnable matrix. The decoder is an auto-regressive LSTM, where the input of every time step is the concatenation of the previous action and the attentive visual feature, $h^t = LSTM([\tilde{f}^t; a^{t-1}], \tilde{h}^{t-1})$. It is desirable that the agent focus on the right part of the instruction throughout the navigation process. Hence we calculate the attentive instruction with the weight calculated over all words of instruction, $\tilde{u}^t = \sum_i^l \beta_i^t u_i, \beta_i^t = softmax_i(u_i W_u h^t)$, where W_u is a learnable matrix. The hidden state is calculated as, $\tilde{h}^t = tanh(W_h(\tilde{u}^t; h^t))$. This instruction-aware hidden state is passed to next time step $t+1$ and used for computing the action distribution at current time step t:

$$P^t(\{a_k^t\}_{k=1}^{m}) = softmax_k(g_k^t W_a \tilde{h}^t) \tag{2}$$

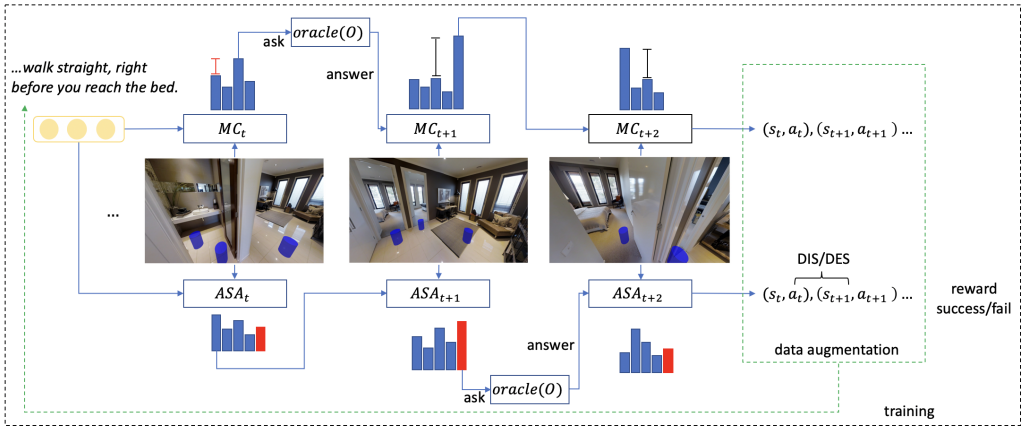


Figure 2: The upper one is the *MC* agent. If the difference of the top2 action probability is less than a threshold, it asks a question and the oracle answers it. The lower one is the *ASA* agent. If the agent picks the *ask* (red bin), it sends a signal to the oracle for help. After the agent is trained, we can run it in the unseen environment, collecting trajectories as augmented data.

g_k^t is calculated as the same as Eq. (1) but on the next navigable locations.

4.2 Optimization

Whenever the agent goes to a viewpoint v_t at time step t, the environment computes the shortest path from v_t to the target location v_n, $\tilde{R}_t = \{v_t, v_{t+1} \cdots v_n\}$. The action going from v_t to v_{t+1} is returned as the teacher action for supervision signal. For the supervised learning loss, we calculate the cross entropy between the teacher action and P^t. For RL, the agent samples its actual action from the action distribution P^t computed in Eq. (2). As in previous work, a +2 reward is given if the final location is within 3 meters distance to v_n. Otherwise, the reward given is -2. We use the advantage actor critic (A2C) Konda and Tsitsiklis [2000] as our RL algorithm.

5 Methodologies

We introduce two agents as in Figure 2 of different levels for the human-agent interaction along with the data augmentation strategy that makes use of the interaction data.

5.1 Training

Model Confusion (MC) In our *MC* model, the idea is that if the agent is confident of itself, then the predicted action distribution should be sharp. To quantify the confusion intuition, we first sort action probabilities in a decreasing order, and say an agent is *confused* if the difference of the top two actions is less than a threshold ϵ:

$$p^t_{sorted}[0] - p^t_{sorted}[1] < \epsilon, \tag{3}$$

we provide the agent with the shortest path action. The threshold is used to control the degree of confusion.

In this simple mode, since the timing of whether to ask questions is not trained, we use the original action space without *ask*. Note that this method can be applied directly on pre-trained models described in sec. 4 during test time.

Action Space Augmentation (ASA) We introduce as many actions as the types of questions the agent asks in addition to the original action space. In this work, the action space is enlarged by 1 to represent the "What should I do next?" question, which is used to indicate whether to ask for help. Formally, the new action space is $\{a_k^t\}_{k=1}^m \cup ask$, where ask is the question indicator. If the agent chooses the ask action, it will remain in the same state s_t and O will give it the action on the shortest path route to v_n. Each selected ask is associated with a negative reward r_{ask}, such that $r_{ask} < 0$ to ensure only necessary questions are asked. The action probability becomes:

$$p^t(\{a_k^t\}_{k=1}^{m+1}) = softmax_{k \cup ask}(g_k^t W_a \tilde{h}^t \cup askW_a \tilde{h}^t) \tag{4}$$

We represent the action feature of *ask* by a vector of dimension same as g_k^t consisting of all ones. Enlarging the action space alone does not provide the desired question-asking behavior without the

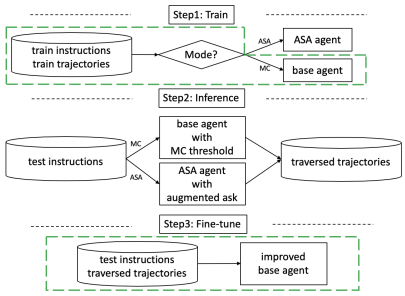


Figure 3: This is the data augmentation process for the *ASA* and *MC* agents. The green boxes indicate the training procedure in Tan et al. [2019]. Agents are not allowed to ask questions at step 3. Finally, the agent is tested for generalizability with different trajectories and instructions (T_b) from the input (T_a) of step 3. $T_{a,b}$ are described in sec. 6.4.

	human-guided exploration	**pre-exploration**
instructions	real, user-specific	fake, user-agnostic
trajectories	not shortest, traversed	shortest, sampled

Table 1: The differences between the human-guided exploration versus pre-exploration.

help from reward shaping Ng et al. [1999], which is a useful technique in training RL algorithms. The difference of distance to the goal location between two consecutive steps can be used as the shaping reward Wang et al. [2019]. However, this will encourage the agent to bias toward the shortest path, but not follow the path indicated by the instruction. We propose to use an additional reward shaping term, the **deviation shaping**, to encourage the agent to follow the instruction. We call the original shaping reward as the distance shaping, $DIS_t = d(v_t, v_n) - d(v_{t+1}, v_n)$, where $d(v_t, v_n)$ is the distance between the current location and the target location. The proposed deviation shaping is, $DEV_t = -d(v_{t+1}, v_{gt})$. v_{gt} is the viewpoint with the shortest distance to v_t among the whole ground truth trajectory. It is straightforward to see that if the agent follows the ground truth trajectory better, the reward should be higher. Moreover, this shaping reward helps reduce the number of questions asked while preserving the same success rate. The reason is better alignment with ground truth trajectories leads to less ambiguities during the navigation. Without *DEV*, the agent will ask questions *at every timestep*. These two shaping rewards are summed together during the RL training. Concretely, the critic CR in the A2C algorithm is optimized at every timestep t as:

$$(CR_t - (DEV_t + DIS_t + r_{ask} + G_t))^2, \tag{5}$$

where G_t is the discounted cumulative reward estimated by the monte carlo method Sutton [1998]. We note that the *intrinsic reward* in Wang et al. [2019] shares a similar idea with our work. However, they train a sequence-to-sequence critic where the input is traversed trajectories and output is instruction decoding probabilities. This critic is used to calculate the cycle reconstruction reward at every timestep, which is much slower than our simple but effective deviation shaping.

Agent Types		success rate		number of questions		move steps		ask percentage	
		val_seen	val_unseen	val_seen	val_unseen	val_seen	val_unseen	val_seen	val_unseen
Base Model w/o Interaction		0.551	0.471	-	-	5.09	4.95	-	-
MC	$\epsilon = 0.1$	0.614	0.562	0.58	0.56	5.01	4.88	0.103	0.103
	0.2	0.693	0.633	1.00	0.96	5.01	4.88	0.167	0.164
	0.3	0.759	0.695	1.34	1.33	5.03	4.88	0.210	0.214
	0.4	0.822	0.756	1.67	1.68	5.05	4.90	0.248	0.255
	0.5	**0.854**	**0.807**	1.95	1.99	5.05	4.92	0.278	0.287
ASA	$r_{ask} = 0.1$	**0.790**	**0.732**	1.25	1.92	5.39	5.49	0.188	0.259
	0.2	0.690	0.710	0.84	1.59	5.58	5.72	0.131	0.218
	0.3	0.704	0.676	0.74	1.41	5.36	5.42	0.120	0.206
	0.4	0.632	0.598	0.38	0.85	5.29	5.25	0.067	0.139
	0.5	0.590	0.494	0.06	0.15	5.04	4.85	0.012	0.030
Human-Guided Exploration	disjoint†	**0.555**	**0.554**	-	-	5.19	4.91	-	-
	random	**0.565**	**0.649**	-	-	5.18	4.93	-	-
Pre-Exploration†		0.560	0.504	-	-	5.21	5.11	-	-

Table 2: Interactive agents, *MC* and *ASA*, both outperform the baseline without interaction. When ϵ is larger, *MC* tends to ask more questions while *ASA* asks fewer questions when r_{ask} is lower. Finally, the proposed human-guided exploration outperforms the pre-exploration techinque Tan et al. [2019] by 5%. †We use the augmented data with size 1500 for a fair comparison.

C	success rate		number of questions	
	ASA-0.1	MC-0.5	ASA-0.1	MC-0.5
0.0	0.732	0.807	1.92	1.99
0.1	0.731	0.743	2.25	1.99
0.2	0.731	0.688	2.69	1.99
0.3	0.732	0.619	3.55	1.99
0.4	0.722	0.562	4.44	1.94

Table 3: The impact of different noise levels. The 0.1 after ASA indicates the $r_{ask} = 0.1$ in Eq. (5), and the 0.5 after MC indicates the $\epsilon = 0.5$ in Eq. (3).

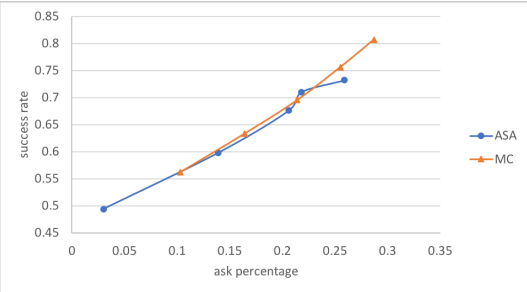


Figure 4: We can only control the threshold or penalty during training, so the points of the two curves would not be plotted on the same x values.

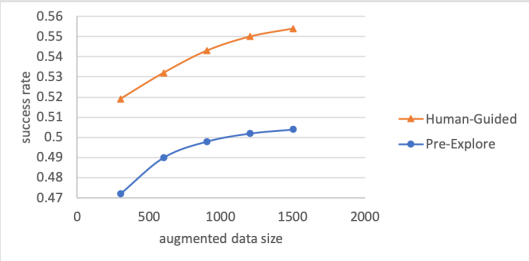


Figure 5: The curves of data augmentation methods. Human-Guided Exploration consistently outperforms Pre-Exploration by a large margin.

5.2 Data Augmentation

Our proposed interaction methods can be used to generate augmented data. Concretely, we execute the trained agent in test environment such that the agent may ask questions and receive answers from O. These trajectories are collected and served as our augmented data. By doing so, human need not to label lots of complete trajectories in advance, but can teach the agent by only answering a few simple questions. The collected data are then used to fine-tune the agent. The complete procedure is outlined in Figure 3. As long as the user keeps using the agent, we can collect more interaction data to fine-tune the agent in a continual learning scenario. The differences between our method, the human-guided exploration, and the pre-exploration approach Wang et al. [2019], Tan et al. [2019] are highlighted in Table 1.

6 Experiments and Discussions

6.1 R2R Data Statistics

In the R2R dataset Anderson et al. [2018], there are 21,567 navigation instructions with an average length of 29 words. The instruction vocabulary consists of around 3100 words due to the nature of the navigation task. The train set includes 61 scenes, with instructions split 14,025 train / 1,020 val seen. 11 scenes and 2,349 instructions are reserved for validating in unseen environments (unseen validation). Please refer to appendix for implementation details.

6.2 Evaluation Metric

We evaluate our agent on the success rate and the number of steps taken which are the standard reported metrics Anderson et al. [2018] of the VLN task. An episode is a success if the navigation error is less than 3 meters. In addition to the standard metrics, we think it is necessary to propose a new evaluation metric to justify the effectiveness of the human-agent interaction, which is the percentage of total actions taken that are *ask* actions, $\frac{\#ask}{\#ask+\#move}$, where $\#move$ is the number of moving actions other than $\#ask$.

6.3 Interaction Results

In this setting, the agent is allowed to ask questions during test time with an oracle providing shortest path actions. For the *ASA* agent, we vary the penalty associated, r_{ask}, to each *ask* action. For the *MC* agent, we adjust the confusion threshold in Eq. (3). Then we test the agent on the unseen validation split. The results are in Table 2.

For the *MC* agent, the success rate and ask percentage become higher when we increase the threshold ϵ. The same observation applies to the *ASA* agent. Note that with a lower penalty r_{ask}, the agent is encouraged to ask more questions. At the same time, the success rate and the ask percentage both increase. If we compare the two agents, the performance is roughly the same regarding the

success rate at the same ratio of the ask actions as shown in Figure 4. It is interesting to note that the move steps of the *ASA* agent increases while the *MC* agent remains the same. We hypothesize this is because *ASA* learns the *ask* behavior during training, so it tries to explore more to maximize the reward. As for *MC*, the threshold is applied only at test time, it does not learn the exploration behavior. While *MC* seems to be simpler and more effective than *ASA*, the *ASA* agent can adapt to errors in human-agent interactions more easily as we will see.

Imperfect Oracle We adjust the distortion probability C in sec. 3.1 to see how our agents adapt to different levels of noise. The results are in Table 3. The *ASA* agent asks more questions with the same success rate while the *MC* agent asks the same number of questions but the success rate drops linearly. The behavior of *ASA* is much more ideal, since it can adjust dynamically to different levels of noise with the same success rate, which is particularly useful if the agent is a real product.

6.4 Human-Guided Exploration

It is desirable that the agent can further improve its navigation ability after several rounds of interactions. We split the unseen validation data to T_a and T_b. This is testing the real use case when a user brings the agent to a new environment (T_a) and teaches it through interaction for a while. After that, the agent is evaluated in the same environment with different instructions and paths (T_b) to see the effectiveness.

Disjoint Split In this setting, T_a and T_b do not share the same trajectories and instructions but the house plans are shared. The reason for this splitting strategy is to compare fairly with the pre-exploration technique since Tan et al. [2019], Wang et al. [2019] ensured the augmented paths are different from those in the test environment. We run the same experiment with *ASA* and *MC* agents on T_a and obtain the interaction history data, which is used to fine-tune the agent. Finally, we test the re-trained agent on T_b without human-agent interaction. There are 2349 instructions in the unseen environment. We use the first 1500 as T_a and the remaining 849 as T_b.

Random Split In this setting, T_a and T_b share the same trajectories but with different instructions. The motivation is to mimic the real-world scenario where customers buy the robot and put it in their houses. It is natural for a human to use different sentences to express the same goal. We randomly permute the unseen validation split and use the first 1500 as T_a and the remaining 849 as T_b.

The results of two settings are in Table 2. We use the best *ASA* ($r_{ask} = 0.1$) agent to generate augmented data. Better performance is observed on the random split setting because the agent has already seen some trajectories in T_a. As for the disjoint setting, we can compare the results of *ASA* agent fairly with Tan et al. [2019]. With the same amount of augmented data (1500 on T_a), our method outperforms theirs by 5% (0.554 vs. 0.504).

Finally, we limit the fine-tune stage (stage 3 in Figure 3) to only supervised training instead of the mixture of supervised and RL training. This is to further reduce the time and energy consumption in test environment which in real-life would be a new customer's home. The result is 0.514 vs. 0.483. While the performance of both methods drops due to the lack of exploration, ours still outperforms the baseline by 3%.

Data Efficiency We vary the augmented data size of the pre-exploration approach to see its data efficiency. The results are in Figure 5. It is easy to see that human-guided exploration can reach the same performance by using much less data, demonstrating that our agent is more data-efficient. Moreover, this experiment highlights the importance of real instructions and trajectories as pointed out in Table 1.

7 Conclusion

In this paper, we propose an interactive learning framework to make the agent capable of resolving ambiguous situations by interacting with a human during the learning or the execution time. Two approaches are proposed, model confusion-based (MC) and RL with reward shaping (ASA). The experiment results demonstrate our agents can strike a balance between the task success rate and number of questions asked. Moreover, the RL agent can adapt dynamically to the noisy oracle. Finally, we propose a strategy to fine-tune the agent using augmented data collected from human-agent interactions, which is more data-efficient and realistic than the previous method.

As for future work, adding more types of questions is a natural next direction. In addition, we will explore changing the interaction form from agent actions to short natural language instructions which is a more realistic interaction format.

References

Peter Anderson, Qi Wu, Damien Teney, Jake Bruce, Mark Johnson, Niko Sünderhauf, Ian Reid, Stephen Gould, and Anton van den Hengel. Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3674–3683, 2018.

Matt MacMahon, Brian Stankiewicz, and Benjamin Kuipers. Walk the talk: Connecting language, knowledge, and action in route instructions. *Def*, 2(6):4, 2006.

David L Chen and Raymond J Mooney. Learning to interpret natural language navigation instructions from observations. In *Twenty-Fifth AAAI Conference on Artificial Intelligence*, 2011.

Valts Blukis, Dipendra Misra, Ross A Knepper, and Yoav Artzi. Mapping navigation instructions to continuous control actions with position-visitation prediction. *arXiv preprint arXiv:1811.04179*, 2018.

Howard Chen, Alane Suhr, Dipendra Misra, Noah Snavely, and Yoav Artzi. Touchdown: Natural language navigation and spatial reasoning in visual street environments. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 12538–12547, 2019.

Xin Wang, Qiuyuan Huang, Asli Celikyilmaz, Jianfeng Gao, Dinghan Shen, Yuan-Fang Wang, William Yang Wang, and Lei Zhang. Reinforced cross-modal matching and self-supervised imitation learning for vision-language navigation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 6629–6638, 2019.

Chih-Yao Ma, Zuxuan Wu, Ghassan AlRegib, Caiming Xiong, and Zsolt Kira. The regretful agent: Heuristic-aided navigation through progress estimation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 6732–6740, 2019a.

Liyiming Ke, Xiujun Li, Yonatan Bisk, Ari Holtzman, Zhe Gan, Jingjing Liu, Jianfeng Gao, Yejin Choi, and Siddhartha Srinivasa. Tactical rewind: Self-correction via backtracking in vision-and-language navigation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 6741–6749, 2019.

Daniel Fried, Ronghang Hu, Volkan Cirik, Anna Rohrbach, Jacob Andreas, Louis-Philippe Morency, Taylor Berg-Kirkpatrick, Kate Saenko, Dan Klein, and Trevor Darrell. Speaker-follower models for vision-and-language navigation. In *Advances in Neural Information Processing Systems*, pages 3314–3325, 2018.

Hao Tan, Licheng Yu, and Mohit Bansal. Learning to navigate unseen environments: Back translation with environmental dropout. *arXiv preprint arXiv:1904.04195*, 2019.

Manolis Savva, Abhishek Kadian, Oleksandr Maksymets, Yili Zhao, Erik Wijmans, Bhavana Jain, Julian Straub, Jia Liu, Vladlen Koltun, Jitendra Malik, et al. Habitat: A platform for embodied ai research. *arXiv preprint arXiv:1904.01201*, 2019.

Chih-Yao Ma, Jiasen Lu, Zuxuan Wu, Ghassan AlRegib, Zsolt Kira, Richard Socher, and Caiming Xiong. Self-monitoring navigation agent via auxiliary progress estimation, 2019b. URL `https://openreview.net/forum?id=r1GAsjC5Fm`.

Michael A Goodrich, Alan C Schultz, et al. Human–robot interaction: a survey. *Foundations and Trends® in Human–Computer Interaction*, 1(3):203–275, 2008.

Luís Seabra Lopes and António Teixeira. Human-robot interaction through spoken language dialogue. In *Proceedings. 2000 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2000)(Cat. No. 00CH37113)*, volume 1, pages 528–534. IEEE.

Dimitris Spiliotopoulos, Ion Androutsopoulos, and Constantine D Spyropoulos. Human-robot interaction based on spoken natural language dialogue.

Terrence Fong, Charles Thorpe, and Charles Baur. Collaboration, dialogue, human-robot interaction. In *Robotics Research*, pages 255–266. Springer, 2003.

Jesse Thomason, Aishwarya Padmakumar, Jivko Sinapov, Nick Walker, Yuqian Jiang, Harel Yedidsion, Justin Hart, Peter Stone, and Raymond J Mooney. Improving grounded natural language understanding through human-robot dialog. *arXiv preprint arXiv:1903.00122*, 2019a.

Jesse Thomason, Michael Murray, Maya Cakmak, and Luke Zettlemoyer. Vision-and-dialog navigation. *CoRR*, abs/1907.04957, 2019b. URL `http://arxiv.org/abs/1907.04957`.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Improving neural machine translation models with monolingual data. *arXiv preprint arXiv:1511.06709*, 2015.

Guillaume Lample, Alexis Conneau, Ludovic Denoyer, and Marc'Aurelio Ranzato. Unsupervised machine translation using monolingual corpora only. *arXiv preprint arXiv:1711.00043*, 2017.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*, 2009.

Vijay R Konda and John N Tsitsiklis. Actor-critic algorithms. In *Advances in neural information processing systems*, pages 1008–1014, 2000.

Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. 1999.

Richard S Sutton. *Introduction to reinforcement learning*, volume 2. 1998.

Geoffrey Hinton, Nitish Srivastava, and Kevin Swersky. Neural networks for machine learning lecture 6a overview of mini-batch gradient descent. 2012.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.

A Implementation Details

Previous works on VLN used ResNet-152 He et al. [2016] pretrained on the ImageNet Deng et al. [2009] to extract 2048-dimensional image feature vectors. We follow Tan et al. [2019] to train the model with the mixture of imitation and RL. In RL training, the discounted factor γ is 0.9. The 256-dimensional word embedding is trained from scratch. The hidden size of the LSTM cell 512, with the maximum decoding length set to 35. For optimizer, RMSprop Hinton et al. [2012] is used with learning rate 10^{-4} and the batch size is 64. Dropout Srivastava et al. [2014] of 0.5 is applied to regularize the model. The global gradient norm is clipped by 40. The *stop* action is represented by an all-zero vector. All agents are trained for 80,000 iterations.